

Exercise 1 — Introduce yourself

Practice: variables, data types, and outputting information.

Create three variables:

- Name: string type.
- Age: int type.
- Is currently learning C#: bool type.

Print to screen:

Name: An

Age: 16

Learning C#: True

Self-check:

- Change the name and age, does the result change correctly?
- If the boolean variable is changed to false, what does the program print?

Self-explain:

- Why is name using string, while age uses int?
- What is the difference between the number 16 and the string "16"?

Hint: use `Console.WriteLine`. This exercise doesn't require keyboard input yet.

Exercise 2 — Calculate notebook cost

Practice: arithmetic operations and order of execution.

A notebook costs 12,000 VND. A customer buys a certain quantity.

Create variables for:

- Price per unit.
- Quantity.
- Total amount.

Calculate and print the total amount.

Quantity	Expected Result
1	12,000
3	36,000
0	0

Requirement: do not hardcode the result 36000 in the print statement. The result must be calculated from the variables.

Extended exercise: add a delivery fee of 5,000 VND, but if the quantity is 0, the total amount is still 0.

Self-explain: when changing the quantity, at which line is the total amount variable recalculated?

Exercise 3 — Validate quantity

Practice: if, else if, else.

The store accepts orders of 1 to 10 notebooks.

Write a program following these rules:

Quantity	Message
Less than or equal to 0	Invalid quantity
From 1 to 10	Can order
Greater than 10	Exceeds limit

Values to test: -1, 0, 1, 10, 11.

Before each run, write down your predicted result.

Self-explain:

- Why do you need to test both 0 and 1?
- If the condition uses < 10 instead of ≤ 10 , what changes?

Exercise 4 — Receive data from the keyboard

Practice: inputting data, type conversion, and handling invalid input.

Upgrade exercise 3: the user inputs the quantity themselves.

The program needs to distinguish:

- Cannot convert the data into an integer.
- Can convert, but the quantity is invalid.
- Valid quantity.

Input Content	Expected Result
abc	Please enter an integer
2.5	Please enter an integer
Empty string	Please enter an integer
0	Invalid quantity
4	Can order
11	Exceeds limit

Hint: research `Console.ReadLine()` and `int.TryParse()`.

Self-explain: does "successfully entered an integer" mean "valid quantity"?

Exercise 5 — Extract a task into a function

Practice: parameters, return values, and return.

Write a function named CalculateTotal:

- Receives the price per unit and the quantity.
- Returns the total amount.
- Do not print the result directly inside the function.

The calling part will receive the result and print it.

Price	Quantity	Returned Result
12,000	3	36,000
5,000	2	10,000
8,000	0	0

In this exercise, assume the input is already a non-negative number.

Self-explain:

- What is a parameter?
- What is the difference between return and Console.WriteLine?
- After the function returns the result, where does the program continue to run?

Hint: before writing the function, write the sentence: "The function receives ... and returns ...".

Exercise 6 — Process multiple quantities

Practice: arrays, loops, and accumulator variables.

Given a list of quantities:

2, 0, 5, -1, 3

Convention for this exercise: a quantity strictly greater than 0 is valid.

Please:

- Print each number.
- Count how many valid quantities there are.
- Calculate the sum of valid quantities.

Expected result:

Valid elements count: 3

Sum of valid quantities: 10

Requirement: use loops, do not use LINQ yet to practice tracking each step.

Create a table on paper:

Current Value	Valid?	Counter After Step	Sum After Step
2	Yes	1	2
0	No	1	2
5	You fill in	You fill in	You fill in
-1	You fill in	You fill in	You fill in
3	You fill in	You fill in	You fill in

Self-explain: why must the sum variable be initialized before the loop?

Exercise 7 — Create class and object

Practice: class, properties, and objects.

Create a Book class with:

- Id: book code.
- Title: book title.
- IsAvailable: whether the book is available to borrow.

Create two different book objects. Borrow the first book by updating IsAvailable, then print the statuses of both.

Expected result: the first book changes its status, the second book remains the same.

Self-explain:

- How does the Book class differ from a specific book object?
- What does new Book() do?
- Can one class create multiple objects?

Exercise 8 — Find book by ID

Practice: list of objects, searching, and null.

Create a list of three books:

ID	Book Title
B01	Learning basic C#
B02	Fun Math
B03	Exploring Nature

Write a FindBookById function:

- Receives an ID.
- Returns the book object if found.
- Returns null if not found.

Calling part:

- Has book → print the book title.

- Does not have → print "Book not found".

Cases to test: B01, B03, B99.

In this exercise, the ID must match exactly, no need to handle case-insensitivity yet.

Self-explain: why do you need to check for null before reading the Title?

Exercise 9 — Assemble into a book borrowing flow

Practice: order of validation, halting execution, and state mutation.

Reuse the list from exercise 8. When the user inputs an ID to borrow:

- Empty ID or all spaces → report "Please enter an ID".
- Book not found → report "Book not found".
- Book is already borrowed → report "The book is currently unavailable".
- Book is available → update the status and report "Successfully borrowed".

Critical test: borrow the same book twice. The first time must succeed, the second time must report it is unavailable.

Self-explain:

- Why check for an empty ID before searching for the book?
- Why search for the book first before checking its status?
- Which line actually mutates the data?

This exercise helps you understand a backend flow: receive data → validate → find object → check business logic → update → return result, but without needing an API or database yet.